

Теоретические основы информатики

Разделы	Темы
Информация и данные	Основные понятия информатики Кодирование информации
Двоичная математика	Системы счисления Битовые операции
Модели и структуры данных	Типы данных Структуры данных
Хранение и обработка информации	Структурированные файлы Парсинг данных Регулярные выражения
Методы информатики	Алгоритмизация Сортировка

Беляков Андрей Юрьевич

к.т.н. , доцент

Тема 6

gitverse.ru/band/TOI

Раздел 3. Модели и структуры данных

Тема 5

Модели данных

- Понятие типа данных.
- Роль типа данных: множество допустимых значений и операции.
- Простые и составные типы данных.
- Структуры (записи) .
- Пользовательские типы данных.

Тема 6

Структуры данных

- Абстрактные структуры данных: стек, очередь, множество, словарь, дерево.
- Интерфейс абстрактных структур данных.
- Реализация алгоритмов с помощью абстрактных структур данных.

Тема 6

Структуры данных

Вопрос 1: Абстрактные структуры данных

Вопрос 2: Механизмы итерации

- интерфейс
- список, массив, стек, очередь
- словарь
- дерево, граф
- итератор, генератор

Вопрос 1

Абстрактные структуры данных



ADT - Abstract Data Type

- это не конкретная реализация (не код)
- это описание механизма доступа к данным
- это способ организации и обработки данных
- в разных задачах удобно по-разному

Зачем нужны и почему их несколько

- переменная-примитив - хранит одно значение
- если значений много - нужна организация
- что именно требуется:
 - часто читать данные
 - часто записывать/менять
 - добавлять/удалять - слева/справа
 - обрабатывать в порядке поступления
 - находить элемент коллекции быстро
 - хранить иерархию
 - хранить уникальные элементы

- универсальное - неоптимально

- под разные задачи лучше
использовать разные виды
структур, коллекций

Структуры данных и коллекции

- **структура данных** - это абстракция
- это модель организации данных
- основа для создания коллекции

- в Python и других языках программирования набор основных структур данных уже реализован в виде классов коллекций

- в Python - list, set, dict, ...

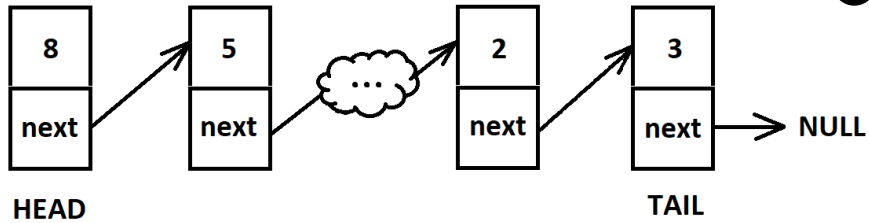
- **коллекция данных** - это конкретная реализация
- в определённом языке программирования
- с формализованным интерфейсом доступа
- с заданными методами работы

Основные виды организации ADT



- 1 **Линейные** (последовательные)
 - Массив (Array)
 - Список (Linked List)
 - Двусвязный список (Doubly Linked List)
 - Стек (LIFO)
 - Очередь (FIFO)
 - Дека (двусторонняя очередь)
- 2 **Контейнерные** (хранение множества)
 - Множество (Set)
 - Словарь (Map / Dictionary)
- 3 **Иерархические**
 - Дерево (Tree)
 - Куча (Heap)
 - Граф (Graph)

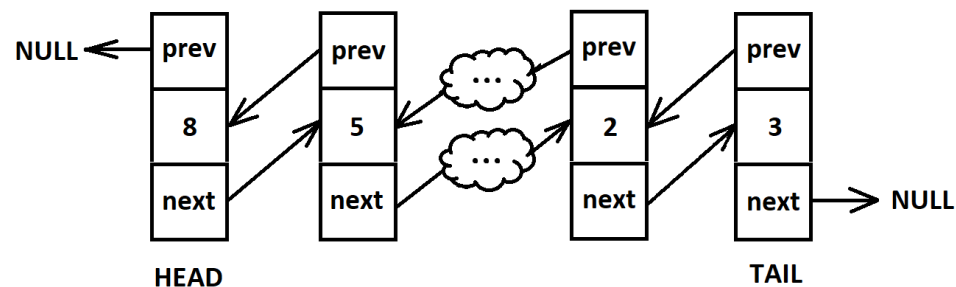
Организация структур данных



массив и список

Список односвязный - в каждом элементе хранит объект и ссылку на следующий элемент
Может хранить разные типы данных
Элементы в памяти разбросаны
Доступ по индексу - $O(N)$ - долго
Размер изменяемый

Список двусвязный - в каждом элементе хранит объект, ссылку на следующий элемент, ссылку на предыдущий элемент



i	0	1	2	3	4
a[i]	2	9	1	8	5

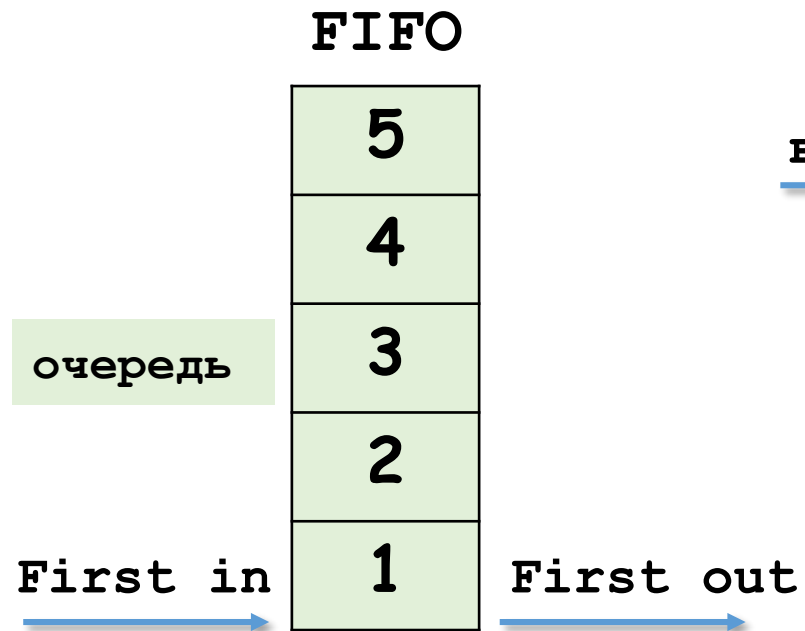
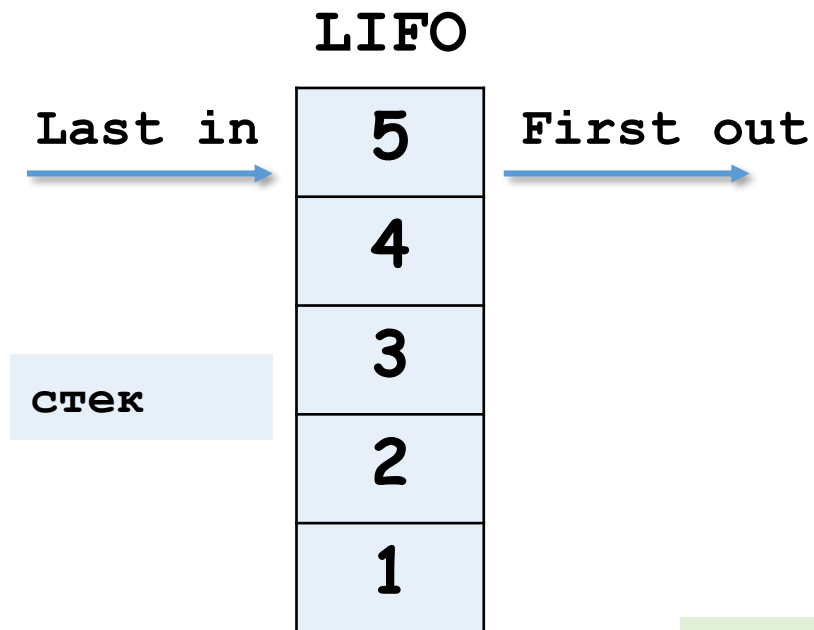
Массив - непрерывный блок памяти
Размер определяется по количеству элементов и по типу данных
Размер неизменяемый
Хранятся данные одного типа и занимают одинаковый размер (2, 4, 8 байтов)
Доступ по индексу - $O(1)$ - очень быстро

Динамический массив - допускает изменение размера, добавляет блоками

стек и очередь

двусторонняя очередь

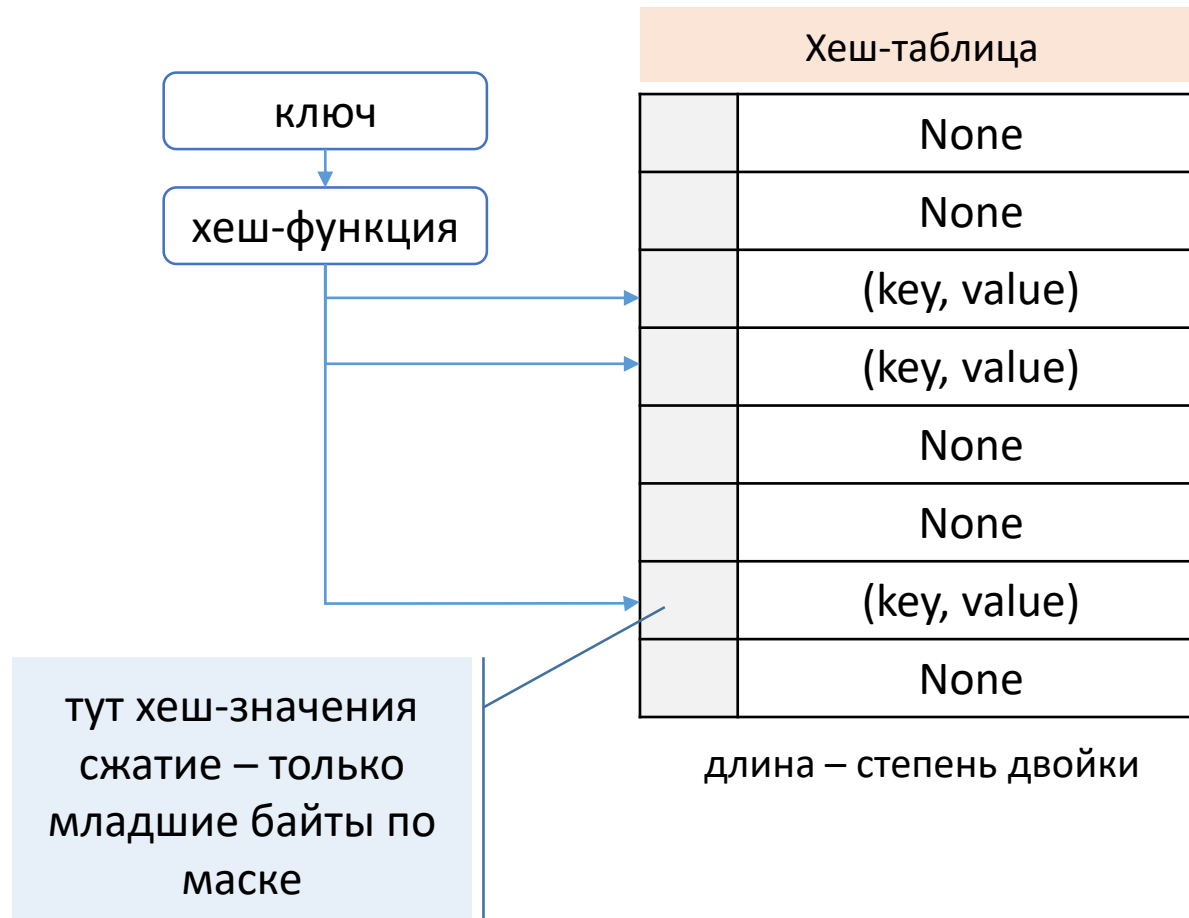
Организация структур данных



множество – это коллекция
уникальных элементов

Организация структур данных

```
print(hash('ABC'))      # 1433972551973331715
print(hash('Python'))   # 2068729341365198690
```



Хеш - это результат преобразования входных данных произвольного размера в битовую строку фиксированной длины с помощью детерминированного алгоритма (хеш-функции).

Детерминированный - если подать одни и те же данные на вход дважды, хеш будет одинаковым

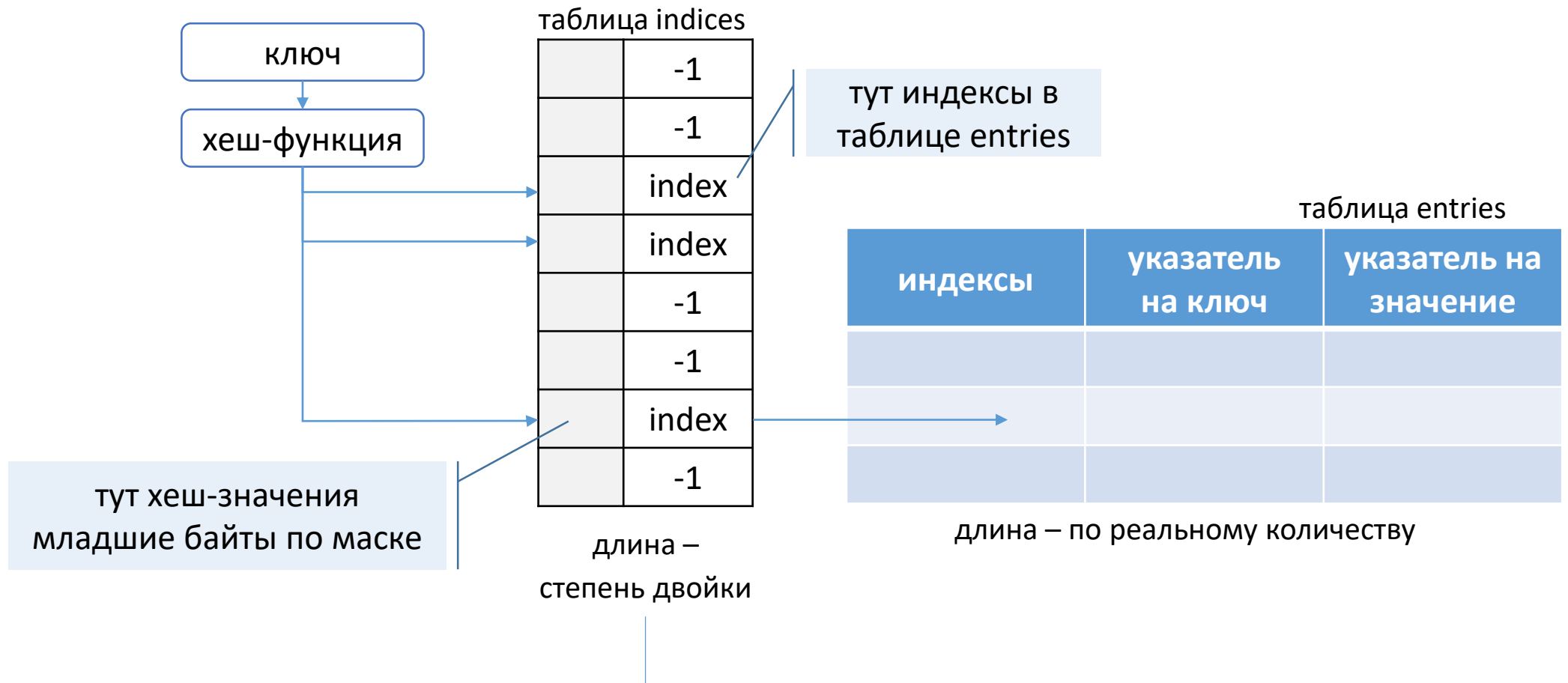
поиск за $O(1)$

при наполненности на 2/3 пересоздаётся, увеличиваясь в 2 раза, поэтому порядок элементов во множестве НЕ гарантирован

словарь – это коллекция
пар ключ-значение

Организация структур данных

поиск за $O(1)$



при наполненности на 2/3 пересоздаётся, увеличиваясь в 2 раза

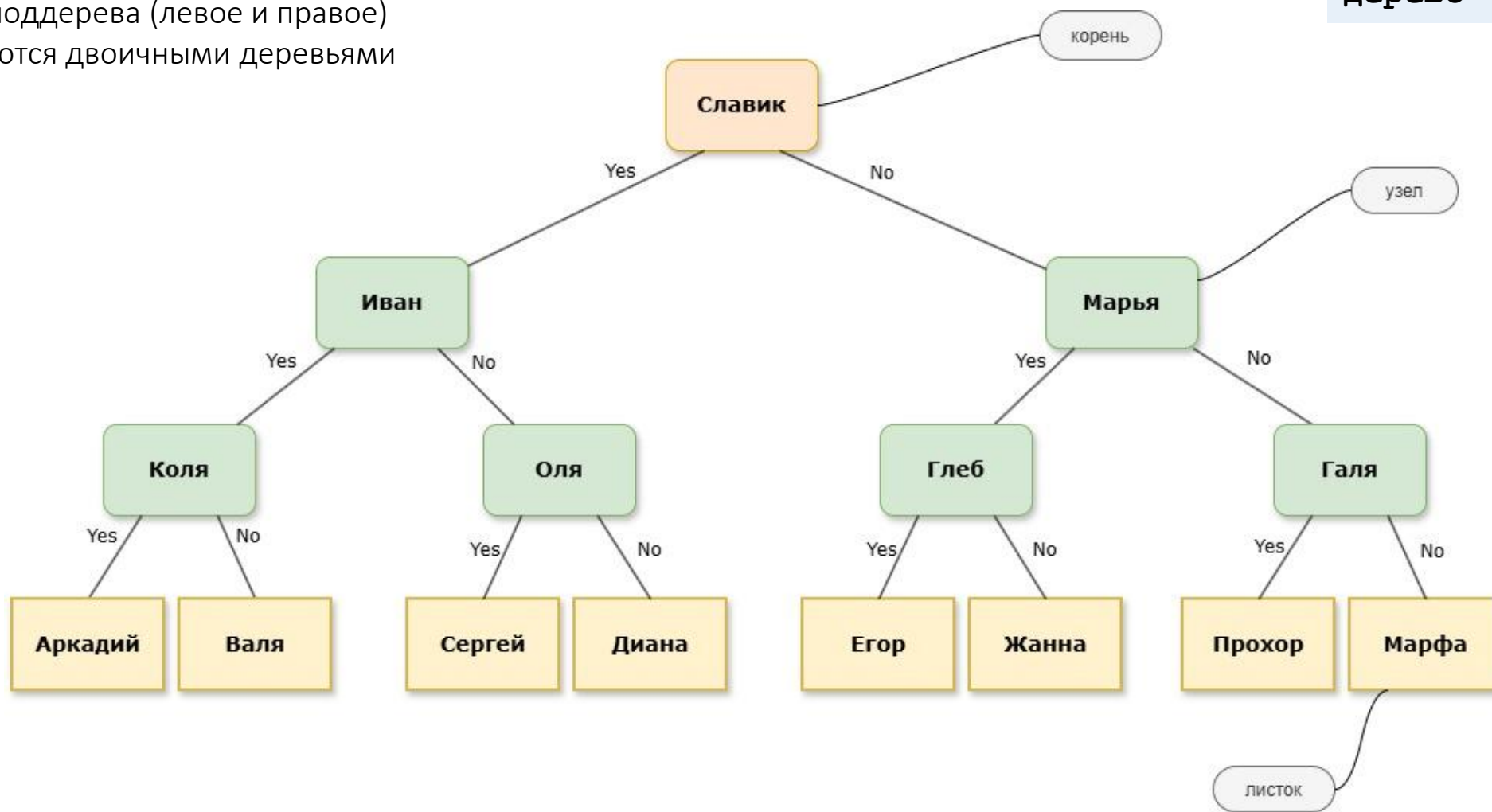
Организация структур данных

Бинарное дерево

для каждого узла выполняется условие:

- оба поддерева (левое и правое)
- являются двоичными деревьями

дерево



Организация структур данных

Бинарное дерево

разместим все элементы дерева в обычный массив,
последовательно собирая сверху-вниз и слева-направо

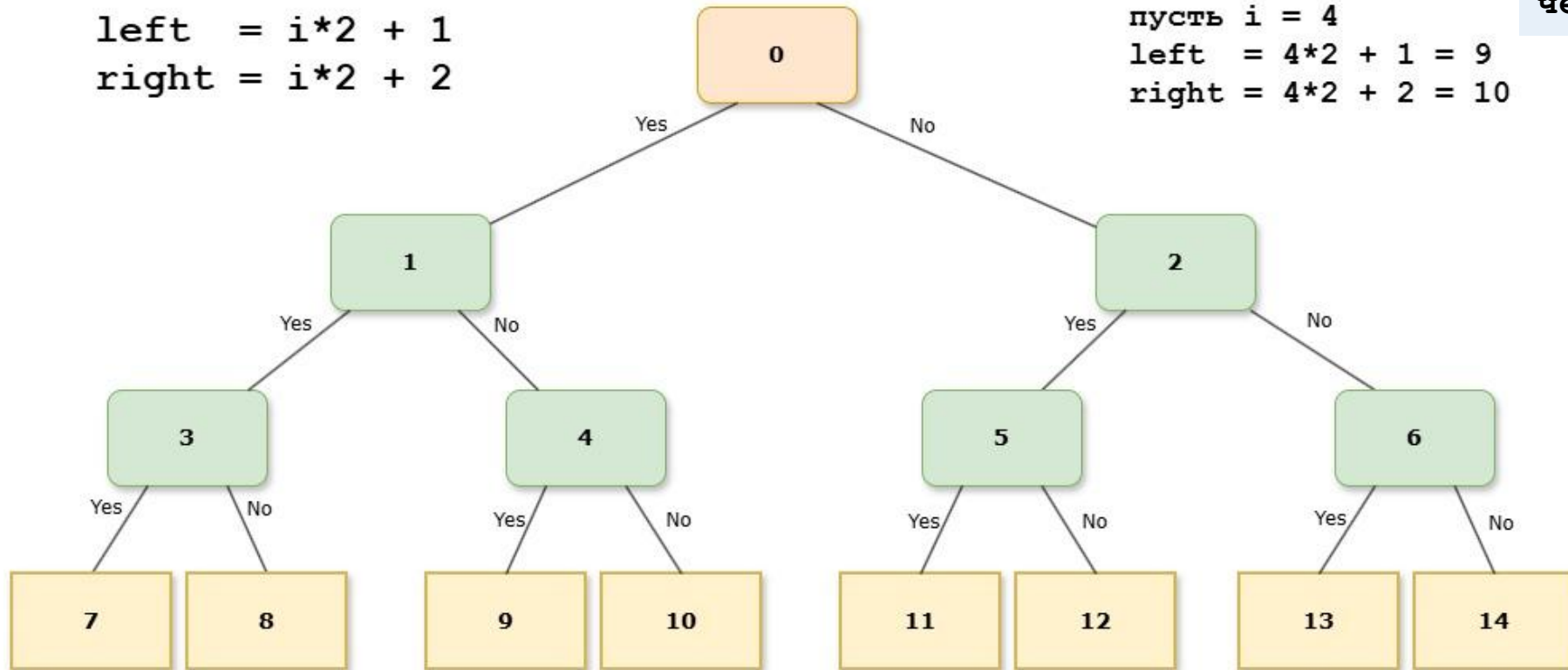
$$\begin{aligned}\text{left} &= i * 2 + 1 \\ \text{right} &= i * 2 + 2\end{aligned}$$

пусть $i = 4$

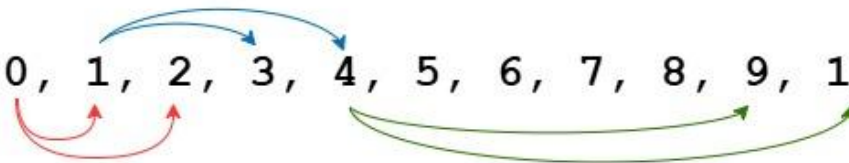
$$\begin{aligned}\text{left} &= 4 * 2 + 1 = 9 \\ \text{right} &= 4 * 2 + 2 = 10\end{aligned}$$

дерево

через массив



array = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]



Бинарное дерево поиска - BST

основное правило:

левое дитя меньше родителя, правое - больше

Организация структур данных

```
class Node:
```

```
    def __init__(self, value):
```

```
        self.key = value
```

```
        self.left = None
```

```
        self.right = None
```

```
# key - вершина (родитель)
```

```
# создаём все узлы
```

```
node1 = Node(1)
```

```
node4 = Node(4)
```

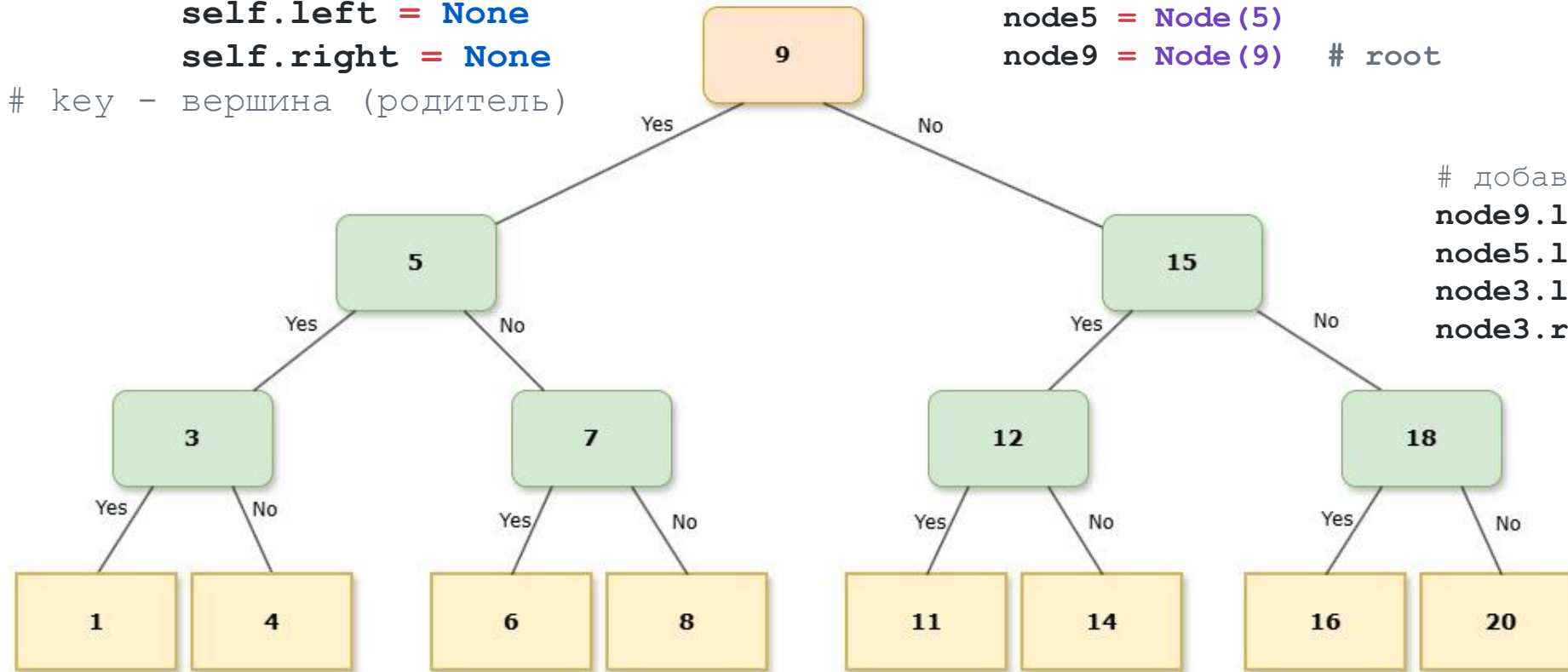
```
node3 = Node(3)
```

```
node5 = Node(5)
```

```
node9 = Node(9) # root
```

дерево - BST

через объекты



```
# добавляем узлы в дерево
```

```
node9.left = node5
```

```
node5.left = node3
```

```
node3.left = node1
```

```
node3.right = node4
```

в дереве (сбалансированном) операции поиска, добавления, удаления за $\log(n)$

высоты левого и правого поддеревьев для каждого узла
отличаются не более чем на 1, то есть высота дерева остается $O(\log n)$

Реализация структур данных

исследуем некоторые реализации в Python

кортеж на 100 млн. займёт 800 Мбайт
только на ссылки + ещё память под объекты

```
tpl = (1, 2)
print(tpl.__sizeof__()) # 48
print(type(tpl)) # <class 'tuple'>
# tpl += (3) # error
# tpl[0] = 9 # error
```

```
lst = [1, 2]
print(lst.__sizeof__()) # 56
print(type(lst)) # <class 'list'>
lst += [3]
print(lst.__sizeof__()) # 104
```

Итого:

24 (заголовок) + 8 (длина) + 8 (ёмкость) + 2 * 8 (указатели) = 24 + 8 + 8 + 16 = **56** байт

24 (заголовок) + 8 (длина) + 8 (ёмкость) + 8 * 8 (указатели) = 24 + 8 + 8 + 64 = **104** байта

Реализация структур данных

коллекции в Python

- кортеж - неизменяемый объект
- размер фиксирован, вычисляется один раз при создании
- состоит из:
 - заголовок объекта: 24 байта (счетчик ссылок + указатель на тип)
 - поле длины коллекции: 8 байт (хранит число элементов)
 - поэтому длина может быть до - $2^{63} - 1$
 - массив указателей: 2 элемента * 8 байт = 16 байт

Итого: 24 + 8 + 16 = **48** байт

- список - изменяемый динамический массив
- он хранит заголовок (24 байта), длину (8 байт),
+ ещё одно поле - выделенная ёмкость (allocated) (8 байт)
- память под указатели выделяется с запасом,
чтобы операция append работала быстро
- Python Runtime определяет сколько памяти выделять
и выделяет с запасом - для будущих добавлений


```
dct = {symbol:ord(symbol)-55 for symbol in 'ABCDEF'}  
print(dct) # {'A': 10, 'B': 11, 'C': 12, 'D': 13, 'E': 14, 'F': 15}
```

```
dct['0'] = 0 # добавить или заменить  
key = 'F'  
print(key in dct) # True
```

```
# .pop() - возвращает val и удаляет key  
print(dct.pop('F')) # 15  
print(key in dct) # False
```

```
for key, val in dct.items():  
    print(key, val)
```

```
st = set()  
st.add(2); st.add(2); st.add(1)  
print(len(st), st) # 2 {1, 2}  
st.discard(2)  
print(len(st), st) # 1 {1}
```

среди встроенных типов Python
нет отдельного класса Stack или Queue

```
stack = []
stack.append(1)      # [1]
stack.append(2)      # [1, 2]
stack.append(3)      # [1, 2, 3]
top = stack.pop()    # [1, 2]
```

```
from collections import deque
```

```
stack = deque()
stack.append(1)      # 1
stack.append(2)      # 1, 2
stack.append(3)      # 1, 2, 3
print(list(stack))   # [1, 2, 3]
print(stack[-1])     # 3
top = stack.pop()    # 3
print(list(stack))   # [1, 2]
print(len(stack))    # 2
while stack: print(stack.pop())
```

Реализация структур данных

коллекции в Python

- для стека - LIFO - последним пришёл, первым ушёл
- проще всего использовать обычный список list
- методы .append() и .pop() реализуют работу с вершиной стека
- можно обсудить алгоритм проверки скобочной последовательности

- для очереди – используем двустороннюю очередь

```
from collections import deque
```

```
queue = deque()

queue.append(1)      # встал в очередь
queue.append(2)
queue.append(3)
print(list(queue))   # [1, 2, 3]
first = queue.popleft() # 1 ушёл
print(len(queue))    # 2
print(list(queue))   # [2, 3]
while queue:
    print(queue.popleft())
```

Реализация структур данных

- добавляется блок примерно на 1/8 от исходного размера

- коллекции в Python

Характеристика	list	array из модуля array	collections.deque
Как устроено	Динамический массив указателей на объекты	Компактный массив значений одного типа	Двусвязный список блоков (не узлов)
Типы элементов	Разнородные	Однородные	Разнородные
Память	Большая 8 байт на ссылку + объекты	Маленькая только данные одного типа	Средняя ссылки + служебные указатели
Доступ по индексу	$O(1)$ мгновенный	$O(1)$ мгновенный	$O(1)$ - в начале/конце $O(n)$ - в середине
Вставка в конец	$O(1)$ амортизированно	$O(1)$ - если есть место $O(n)$ - при перераспределении	$O(1)$ — всегда.
Вставка в начало	$O(n)$ сдвиг всех элементов	$O(n)$ сдвиг всех элементов.	$O(1)$ всегда один
Вставка в середину	$O(n)$ сдвиг	$O(n)$ сдвиг	$O(n)$ обход до середины

- амортизированно — это в среднем за большое количество операций

Контрольные вопросы

Понятие структуры данных и коллекции

Особенности и сравнения абстрактных структур:

- массив и список

- односвязный/двусвязный список

- словарь

- множество

- дерево

Особенности реализации в Python



Вопрос 2

Механизмы итерации

итерация нужна для работы с коллекциями (объектами) :

- осуществлять перебор, поиск, ...

итератор отдаёт элементы по одному (экономит память)

объект поддерживает итерацию,

если для него реализованы методы:

- `__iter__()` - возвращает итератор

- `__next__()` - возвращает след. элемент



Механизм перебора коллекции

```
lst = list('025')  
# ['0', '2', '5']
```

```
# for неявно вызывает iter()  
# затем next() до StopIteration  
for item in lst: print(item)
```

```
it = iter(lst)  
while (item := next(it, None)) is not None:  
    print(item)
```

Задумано, что все
будут делать так

```
it = iter(lst) # создаём итератор  
# вызов метода объекта lst.__iter__()  
  
# явно вызываем next()  
print(next(it))    # 0  
print(next(it))    # 2  
print(next(it))    # 5  
# если вызвать next() ещё раз:  
# print(next(it))  # получим StopIteration
```

```
it = iter(lst)  
while True:  
    try:  
        item = next(it)  
        print(item)  
    except StopIteration:  
        break
```

Механизм перебора коллекции

```
lst = [1, 2, 3]
tpl = (1, 2, 3)
line = "123"
st = {1, 2, 3} # множество (порядок не гарантирован)
dct = {'a': 1, 'b': 2} # словарь (итерация по ключам)
```

```
it = iter(dct)
print(dct[it.__next__()]) # 1
print(dct[it.__next__()]) # 2
for key in dct: print(key, dct[key])
# a 1
# b 2
```

```
import os
```

```
file_name = os.path.basename(__file__)
print(f"Имя файла: {file_name}")
with open(file_name, 'r', encoding='utf8') as f:
    for line in f: # итерация по строкам файла
        print(line, end='')
```

свой класс-итератор - сделаем так
чтобы правая граница входила

```
class MyRange:
    def __init__(self, start, end):
        self.current = start
        self.end = end
    def __iter__(self):
        return self
    def __next__(self):
        if self.current > self.end:
            raise StopIteration
        val = self.current
        self.current += 1
        return val
```

```
my_range = MyRange(1, 4)
print(my_range)
print([elm for elm in my_range])
```

атрибуты - у класса
ключи - у словаря

можно на практическом занятии разобрать способы итерации
атрибутов объекта (класса), включая создание собственного итератора

```
symbols = 'Python'

codes = [ord(symbol) for symbol in symbols]
# [80, 121, 116, 104, 111, 110]

dct = {symbol:ord(symbol) for symbol in symbols}
# {'P': 80, 'y': 121, 't': 116, 'h': 104, 'o': 111, 'n': 110}
```

```
coords_X, coords_Y = (0,1,2), (0,1,2)
w = ((x,y) for x in coords_X for y in coords_Y)
print(w)
# <generator object <genexpr> at 0x0000023D5B06C820>

for pair in w:
    x, y = pair
    dist = (x**2 + y**2)**.5
    print(f'X = {x}; Y = {y}; dist = {dist:.2f}')
```



genexp - это в круглых скобках - итератор

итератор отдаёт элементы по одному (экономит память)

```
X = 0; Y = 0; dist = 0.00
X = 0; Y = 1; dist = 1.00
X = 0; Y = 2; dist = 2.00
X = 1; Y = 0; dist = 1.00
X = 1; Y = 1; dist = 1.41
X = 1; Y = 2; dist = 2.24
X = 2; Y = 0; dist = 2.00
X = 2; Y = 1; dist = 2.24
X = 2; Y = 2; dist = 2.83
```


Инструменты перебора _ _ _ _

```
# генераторное выражение  
gen_1 = (x**2 for x in (1,2,3))
```

```
# функция-генератор  
def _gen():  
    for x in 1,2,3:  
        yield x**2
```

yield

```
gen_2 = _gen()
```

```
print(gen_1)  
# <generator object <genexpr> at 0x0000019CFE521220>  
print(gen_2)  
# <generator object _gen at 0x0000019CFE521700>  
print(list(gen_1)) # [1, 4, 9]  
print(list(gen_2)) # [1, 4, 9]
```

genexpr – неявно использует yield

свой итератор может использовать более сложную логику и множественные yield



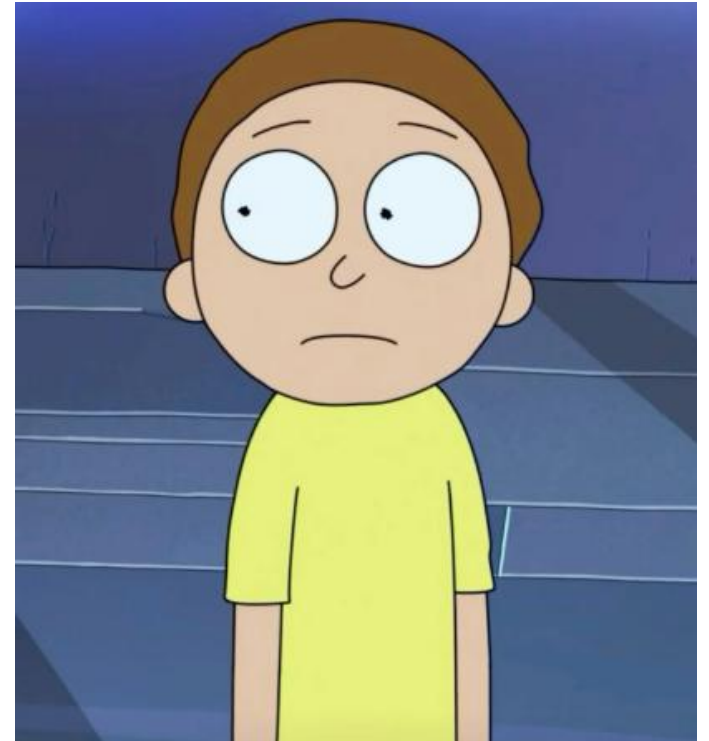
Контрольные вопросы

`listcomp`

`genexp`

функция-генератор

`yield`



Теоретические основы информатики

Раздел 3. Модели и структуры данных

Тема 6: Структуры данных

2026

Беляков Андрей Юрьевич